

TECHBIZ SECURITY

CYBERSECURITY INTERNSHIP REPORT

Week 6 – Task 17 (EH)

PASSWORD CRACKING — HASHES

JOHN THE RIPPER AND HASHCAT

Submitted By	MANOJ
Role	Cyber Security Intern
Internship Organisation	TechBiz Security
Internship ID	TBS-INT-2026-518
Submitted Date	26 September 2026

1. Introduction

Password hashing is a fundamental security control used to protect user credentials within authentication systems. Rather than storing a password directly, a system stores the output of a one-way cryptographic hash function applied to the password. This ensures that the original password is not directly recoverable from the stored value under normal circumstances. However, if a hash is weak, unsalted, or generated by an outdated algorithm, it becomes susceptible to offline password cracking attacks in which an attacker attempts to recover the original password by comparing candidate hashes against the target hash.

Password cracking, when performed in an authorized and controlled environment, is a recognised technique in penetration testing and security auditing. It is used to evaluate the strength of stored credentials and to demonstrate the risks associated with weak passwords and insecure hashing algorithms. This report documents a practical exercise performed as part of Week 6, Task 17 (EH) of the TechBiz Security cybersecurity internship, focusing on password hash preparation and cracking using two industry-standard tools.

Two password cracking tools were used during this exercise: John the Ripper, a widely used offline password cracking utility supporting numerous hash formats, and Hashcat, a high-performance password recovery tool capable of leveraging CPU and GPU resources for large-scale hash cracking operations. In addition to the local Kali Linux practical work, this task also included completion of the "Crack the hash" room on the TryHackMe platform, which provided supplementary hands-on learning related to password hash identification and cracking.

2. Objectives

- Understand password hashes and their role in authentication security.
- Identify the MD5 hash format used in the local practical.
- Use John the Ripper to perform dictionary-based password cracking.
- Verify a recovered password using John the Ripper.
- Use Hashcat to crack and display the recovered password.
- Complete the TryHackMe "Crack the hash" learning activity.
- Understand the security risks associated with weak passwords and weak hashing practices.

3. Lab Environment and Authorized Scope

The practical exercise was carried out within a controlled and authorized lab environment established for cybersecurity learning and internship training purposes. Table 1 summarises the environment and tools used during the exercise.

Parameter	Details
Operating System	Kali Linux
Tools Used	John the Ripper, Hashcat
Wordlist	/usr/share/wordlists/rockyou.txt

Hash Type Demonstrated	MD5
Learning Platform	TryHackMe
TryHackMe Room	Crack the hash

Table 1: Lab environment and tools used during the practical exercise.

All activities described in this report were performed strictly for cybersecurity learning purposes within the authorized internship training environment. No external, third-party, or production systems were targeted at any stage of this exercise.

4. Hash Preparation and Identification

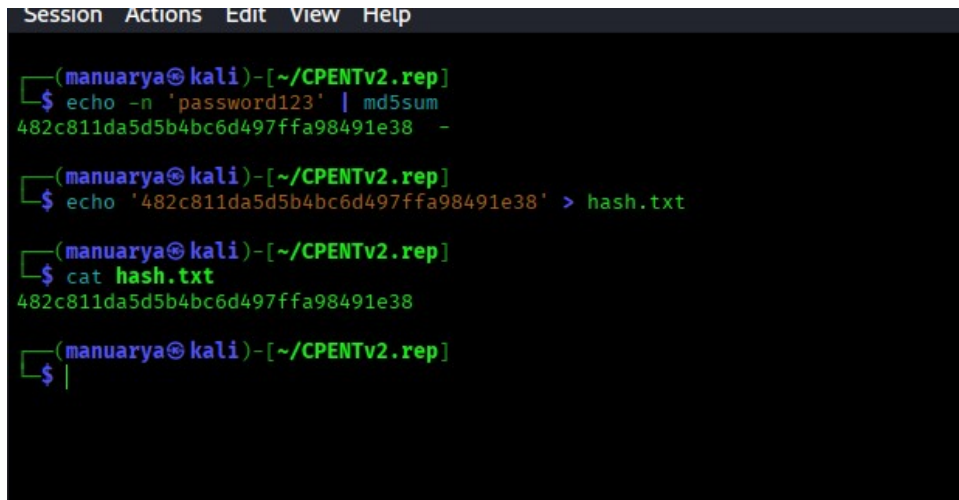
The first stage of the exercise involved generating and preparing an MD5 hash for use in the subsequent cracking demonstration. An MD5 hash was generated from the plaintext string "password123" using the md5sum utility in Kali Linux. The resulting hash value was:

```
482c811da5d5b4bc6d497ffa98491e38
```

This hash was then saved into a file named hash.txt, and the contents of the file were verified using the cat command to confirm that the hash had been stored correctly. Following this, the available John the Ripper hash formats were checked to confirm support for the MD5 format, using the following command:

```
john --list=formats | grep -i md5
```

The output confirmed that John the Ripper supports the MD5 hash format, including the raw-MD5 format required for cracking the prepared hash. This step ensured that the hash was correctly identified and ready for a dictionary-based cracking attack.



```

Session Actions Edit View Help

(manuarya@kali)-[~/CPENTv2.rep]
$ echo -n 'password123' | md5sum
482c811da5d5b4bc6d497ffa98491e38 -

(manuarya@kali)-[~/CPENTv2.rep]
$ echo '482c811da5d5b4bc6d497ffa98491e38' > hash.txt

(manuarya@kali)-[~/CPENTv2.rep]
$ cat hash.txt
482c811da5d5b4bc6d497ffa98491e38

(manuarya@kali)-[~/CPENTv2.rep]
$

```

Figure 1: Preparation and verification of the MD5 hash file in Kali Linux.

5. Password Cracking Using John the Ripper

With the hash prepared and its format confirmed, John the Ripper was used to attempt a dictionary-based password cracking attack against the MD5 hash. The following command was executed:

```
john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
```

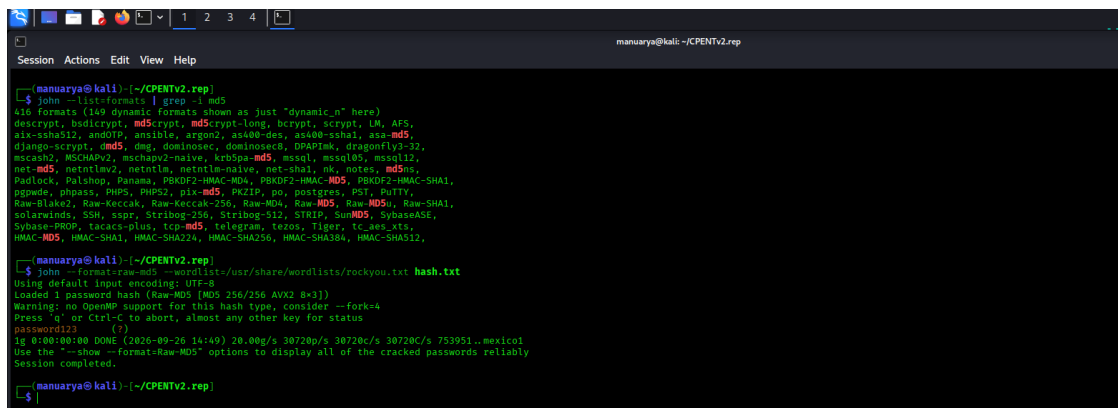
The parameters used in this command are explained below:

- `--format=raw-md5` specifies that the target hash is in the raw MD5 format.
- `--wordlist` specifies the dictionary file used for the attack.
- `/usr/share/wordlists/rockyou.txt` is the wordlist supplied to John the Ripper.
- `hash.txt` is the input file containing the target hash.

As shown in Figure 2, John the Ripper successfully recovered the password associated with the hash. The recovered password was:

```
password123
```

This result demonstrates a successful dictionary-based password cracking attack against a weak password that was protected only by an unsalted MD5 hash. Because the password "password123" is a common value present in the RockYou wordlist, it was recovered almost instantly.



```
manuaria@kali: ~/CPENTv2.rep
$ john --list-formats | grep -i md5
416 formats (149 dynamic formats shown as just "dynamic_n" here)
descript, bsdictcrypt, md5crypt, md5:crypt-long, bcrypt, scrypt, LM, AFS,
aix-ssh512, andotp, ansible, argon2, as400-des, as400-ssha1, asa-md5,
django-crypt, md5, mg, dominsec, dominsec2, DPAPIM, dragonfly-32,
mcsash2, MSCSHAPv2, mschapv2-naive, krb5pa-md5, mssql, mssql05, mssql12,
net-md5, netntlmv2, netntlm, netntlm-naive, net-sha1, nk, notes, md5ms,
Padlock, Palshop, Panama, PBKDF2-HMAC-MD4, PBKDF2-HMAC-MD5, PBKDF2-HMAC-SHA1,
pgpwe, ppass, PPS, PPS2, plx-md5, PzZip, ps, postgres, PST, Putty,
Raw-Blake2, Raw-Keccak, Raw-Keccak-256, Raw-MD4, Raw-MD5, Raw-MD5s, Raw-SHA1,
solarwindy, SSH, sspr, Stribog-16, Stribog-512, STRIP, SunMD5, SybaseASE,
Sybase-MP, taccap-plus, tcp-md5, telegram, 14206, Tiger, tc_ess.txt,
HMAC-MD5, HMAC-SHA1, HMAC-SHA224, HMAC-SHA256, HMAC-SHA384, HMAC-SHA512,
manuaria@kali: ~/CPENTv2.rep
$ john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork-a
Press 'q' or Ctrl-C to abort, almost any other key for status
password123 (7)
1g 8480-86328 DONE (2026-09-26 14:49) 28.00g/s 38728p/s 38728C/s 753951..mexico!
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.
manuaria@kali: ~/CPENTv2.rep
$
```

Figure 2: John the Ripper successfully cracking the MD5 hash using the RockYou wordlist.

6. Verifying the Cracked Password with John the Ripper

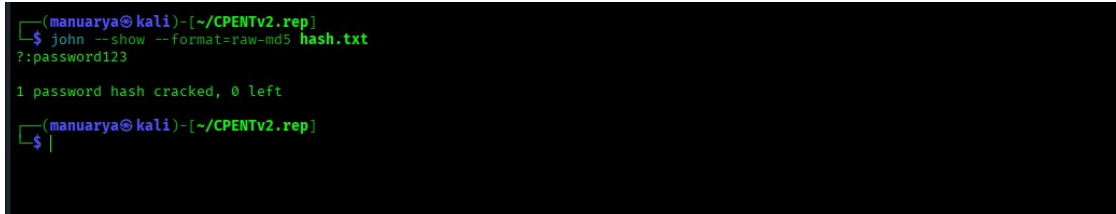
After the password was cracked, John the Ripper's `--show` option was used to verify and display the previously recovered credential associated with the stored hash. The following command was executed:

```
john --show --format=raw-md5 hash.txt
```

As shown in Figure 3, the output confirmed the recovered password as password123, and John the Ripper reported the following cracking status:

```
1 password hash cracked, 0 left
```

This step confirms that the `--show` option is used to display credentials that have already been recovered and stored by John the Ripper, without needing to repeat the cracking process.

A terminal window with a black background and green text. The prompt is (manuarya@kali)~[~/CPENTv2.rep]. The command john --show --format=raw-md5 hash.txt is entered. The output shows the password 'password123' and a status message '1 password hash cracked, 0 left'.

```
(manuarya@kali)~[~/CPENTv2.rep]
$ john --show --format=raw-md5 hash.txt
?:password123

1 password hash cracked, 0 left
(manuarya@kali)~[~/CPENTv2.rep]
$
```

Figure 3: Verification of the recovered password using John the Ripper.

7. Password Cracking Using Hashcat

To complement the John the Ripper demonstration, Hashcat was also used to independently recover and display the password associated with the same MD5 hash. The following command was executed:

```
hashcat -m 0 hash.txt --show
```

The parameters used in this command are explained below:

- -m 0 specifies the MD5 hash mode within Hashcat.
- hash.txt is the input file containing the target hash.
- --show displays the previously recovered password stored in Hashcat's results, rather than repeating the cracking process.

As shown in Figure 4, Hashcat displayed the recovered credential in the following format:

```
482c811da5d5b4bc6d497ffa98491e38:password123
```

The Hashcat session output also confirmed the following status details:

- Status: Cracked
- Hash.Mode: 0 (MD5)
- Recovered: 1/1
- The RockYou wordlist (/usr/share/wordlists/rockyou.txt) was used during the cracking process.

```

manuarya@kali
Session Actions Edit View Help
* Single-Hash
* Single-Salt
* Raw-Hash

ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 90c

Host memory allocated for this attack: 513 MB (4197 MB free)

Dictionary cache hit:
* Filename ..: /usr/share/wordlists/rockyou.txt
* Passwords..: 14344385
* Bytes.....: 139921507
* Keyspace...: 14344385

482c811da5d5b4bc6d497ffa98491e38:password123

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 0 (MD5)
Hash.Target.....: 482c811da5d5b4bc6d497ffa98491e38
Time.Started.....: Sat Sep 26 14:53:17 2026 (0 secs)
Time.Estimated...: Sat Sep 26 14:53:17 2026 (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#01.....: 746.4 kH/s (0.46ms) @ Accel:1024 Loops:1 Thr:1 Vec:8
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 4096/14344385 (0.03%)
Rejected.....: 0/4096 (0.00%)
Restore.Point...: 0/14344385 (0.00%)
Restore.Sub.#01...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#01...: 123456 -> oooooo
Hardware.Mon.#01.: Util: 25%

Started: Sat Sep 26 14:52:48 2026
Stopped: Sat Sep 26 14:53:18 2026

(manuarya@kali) [~/CPENTV2.rep]
$ hashcat -m 0 hash.txt --show
482c811da5d5b4bc6d497ffa98491e38:password123

(manuarya@kali) [~/CPENTV2.rep]
$

```

Figure 4: Hashcat successfully recovering and displaying the MD5 password.

8. TryHackMe Practical — Crack the Hash

In addition to the local Kali Linux practical work described in the preceding sections, the exercise also included a hands-on learning activity on the TryHackMe platform. The room accessed for this activity was titled "Crack the hash".

As shown in Figure 5, the room progress indicator confirms that the room was completed, with the platform displaying "Room completed: 100%". This report does not include individual TryHackMe answers, flags, hashes, or commands used within the room, as these are not visible in the supplied evidence.

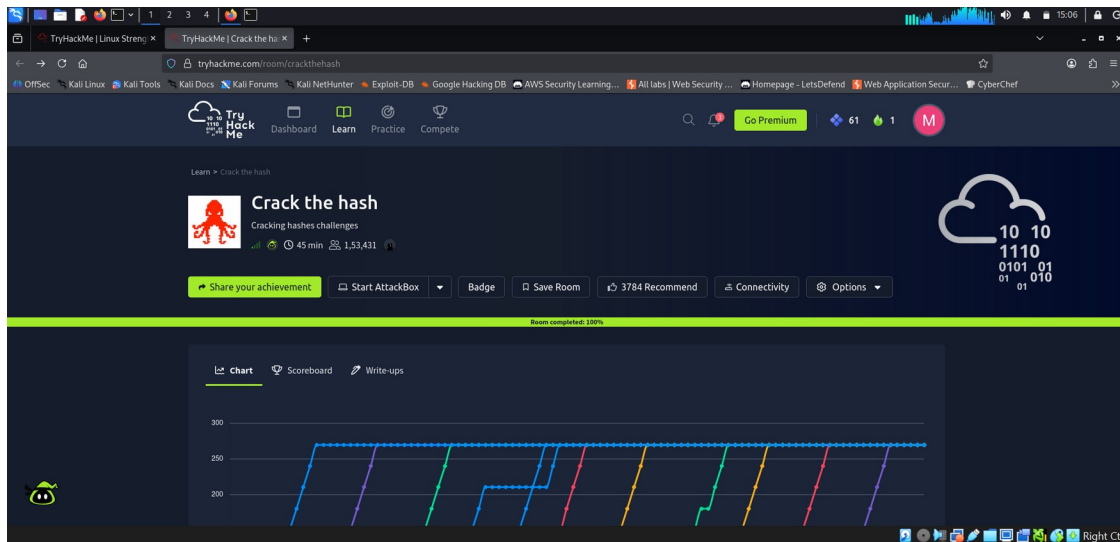


Figure 5: Completion of the TryHackMe Crack the hash room.

It is important to note that the TryHackMe activity was performed on TryHackMe's own dedicated learning infrastructure and is distinct from the local practical work carried out on the Kali Linux virtual machine documented in Sections 4 to 7 of this report.

9. Technical Analysis

9.1 MD5

MD5 (Message Digest Algorithm 5) is a cryptographic hash function that produces a 128-bit hash value. While historically used for data integrity checks and password storage, MD5 is now considered unsuitable for secure password storage. This is because MD5 is computationally fast, which allows an attacker to test an extremely large number of password candidates per second during an offline cracking attempt. Additionally, MD5 does not incorporate salting by default, meaning identical passwords will always produce identical hash values, further increasing susceptibility to precomputed and dictionary-based attacks.

9.2 Dictionary Attacks

A dictionary attack is a password cracking technique in which candidate passwords are drawn from a predefined wordlist and hashed using the same algorithm as the target hash. Each resulting hash is compared against the target hash, and a match indicates that the correct password has been found. Dictionary attacks are highly effective against passwords that are common, predictable, or previously exposed in known data breaches, as such passwords are likely to appear within widely used wordlists such as RockYou.

9.3 John the Ripper

John the Ripper is a widely used password auditing and cracking tool that supports a large number of hash formats, including raw MD5, as confirmed during the hash identification stage of this exercise. It allows a large number of password candidates to be tested efficiently against a target hash and provides functionality to store, display, and verify previously cracked credentials.

9.4 Hashcat

Hashcat is a high-performance password recovery tool that supports a wide range of hashing algorithms and attack modes, including dictionary, brute-force, mask, and rule-based attacks. Hashcat is capable of leveraging both CPU and GPU processing resources, which allows it to test very large numbers of password candidates in a short period of time, making it a widely used tool for password auditing in authorized security assessments.

9.5 Weak Passwords

The password used in this exercise, password123, is a simple and commonly used password that appears within widely available password wordlists such as RockYou. As demonstrated in Sections 5 and 7, this allowed both John the Ripper and Hashcat to recover the password almost instantly. This highlights how quickly weak or common passwords can be compromised when they are protected only by a fast, unsalted hashing algorithm such as MD5.

10. Security Risks

This practical exercise highlights several security risks associated with weak password choices and outdated hashing practices:

- Weak or commonly used passwords can be recovered quickly through dictionary attacks.
- Fast legacy hashing algorithms, such as MD5, provide poor resistance to offline cracking attempts.
- Reuse of common passwords across multiple accounts increases the risk of widespread account compromise.
- Exposed password hashes can become a serious security issue if appropriate password hashing protections are not implemented.
- Organizations should not rely on outdated hashing algorithms for password storage, as this significantly weakens their overall authentication security posture.

11. Mitigation and Security Recommendations

Based on the risks identified during this exercise, the following defensive recommendations are proposed:

- Do not use MD5 for password storage.
- Use modern password hashing algorithms designed specifically for password storage, such as Argon2id, bcrypt, scrypt, or PBKDF2 where appropriate.
- Use unique, randomly generated salts for each stored password.
- Enforce strong password policies covering minimum length and complexity requirements.
- Encourage the use of unique passwords across different accounts and systems.
- Implement multi-factor authentication to reduce reliance on passwords alone.
- Protect password databases and restrict access to authorized personnel only.
- Monitor authentication systems for suspicious or anomalous login activity.
- Conduct authorized password audits periodically to identify weak or compromised credentials.

- Never store passwords in plaintext under any circumstances.

12. Learning Outcomes

The following learning outcomes were achieved through completion of this practical exercise:

- Understanding of password hashes and their role in authentication security.
- Understanding of MD5 hash identification using John the Ripper's format listing functionality.
- Practical experience using John the Ripper for dictionary-based password cracking.
- Practical experience using Hashcat for hash recovery and result verification.
- Improved understanding of the importance of strong, unique passwords.
- Understanding of why legacy password hashing algorithms, such as MD5, are unsuitable for secure password storage.
- Practical experience gained through completion of TryHackMe's Crack the hash room.

13. Conclusion

This practical exercise demonstrated how weak passwords, when combined with unsuitable hashing algorithms such as MD5, can be readily compromised through offline password-cracking techniques. Using John the Ripper and Hashcat within an authorized Kali Linux lab environment, the password protected by the prepared MD5 hash was successfully recovered and independently verified using both tools. In addition, the TryHackMe "Crack the hash" room was completed, providing further hands-on exposure to password hash cracking concepts.

This exercise was conducted strictly within an authorized internship training environment for learning purposes. It does not represent professional penetration-testing experience against production systems, nor does it constitute or claim any real-world security compromise.

14. Evidence Summary

Figure	Evidence	Description
Figure 1	Hash Preparation	MD5 hash generated from "password123" and saved to hash.txt
Figure 2	John the Ripper	MD5 hash cracked using the RockYou wordlist
Figure 3	John Verification	Recovered password displayed using john --show
Figure 4	Hashcat	MD5 hash shown as successfully cracked, recovered 1/1
Figure 5	TryHackMe	Crack the hash room completed at 100%

15. References

- John the Ripper documentation — <https://www.openwall.com/john/doc/>
- Hashcat documentation — <https://hashcat.net/wiki/>

- TryHackMe, "Crack the hash" room — <https://tryhackme.com/room/crackthehash>
- OWASP Password Storage Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html